Visual Maze-Solving Agent

Two Architectures · Two Failure Modes

2026. 3. 18 (Wed.)

Dohoon Kim

MIDP, Duke University

The question

What happens when you build the same agent two ways,

once with classical methods, once with deep learning?

CLASSICAL

OpenCV perception + BFS planning

Hypothesis: brittle, will fail under visual noise.

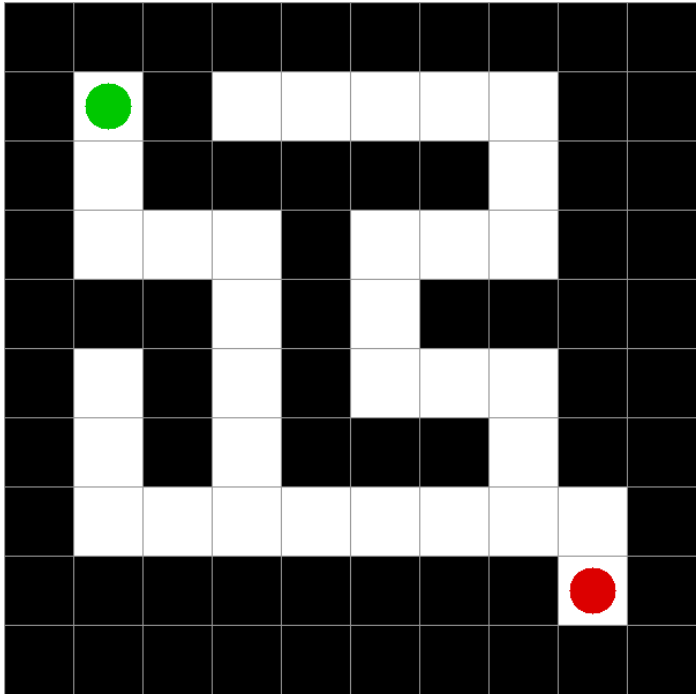
DEEP LEARNING

Claude Vision + Behavior-Cloned MLP

Hypothesis: robust, will generalize across conditions.

Both hypotheses were wrong, in different ways.

Task: solve a maze from its image



10 × 10 grid · black = walls · green = start · red = goal

PERCEPTION

Extract grid structure from pixels

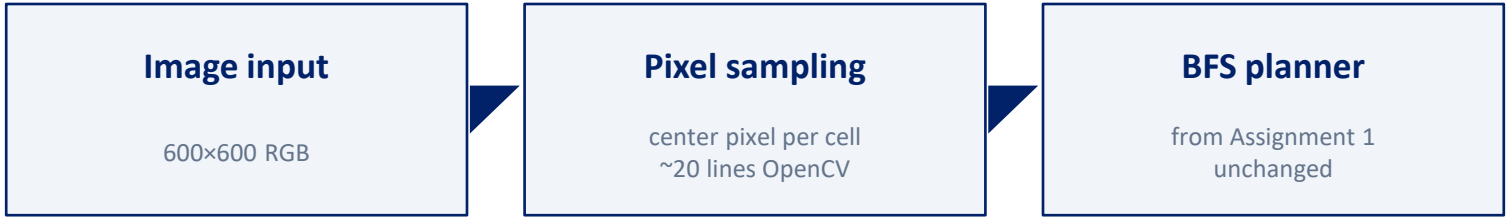
PLANNING

Find action sequence from start → goal

CONTROL

Execute plan & verify legality

Non-DL agent



DESIGN RATIONALE

Minimal by choice. Want to see what classical actually provides — without heuristics propping it up.

Trade-off. Fast (milliseconds), deterministic, fully interpretable — three properties that are hard to match.

Continuity. BFS planner is the same code from Assignment 1. Grid navigation maps cleanly onto the state-action formulation.

DL agent

DESIGN RATIONALE

Zero-shot perception. Frontier VLM solves the task without any training data or GPU time.

Behavior cloning over DQN. Pure supervised learning on optimal BFS trajectories — no reward shaping, no replay buffer.

Trade-off. ~1000× slower per inference; API call adds non-determinism. Policy only knows states seen in training.

Why not the alternatives?

QUESTION

Why not train a custom CNN for perception?

A CNN would need dataset curation, augmentation, and training time.

A VLM solves it zero-shot.

The trade-off: training complexity swapped for API latency and cost.

QUESTION

Why not DQN or RL for planning?

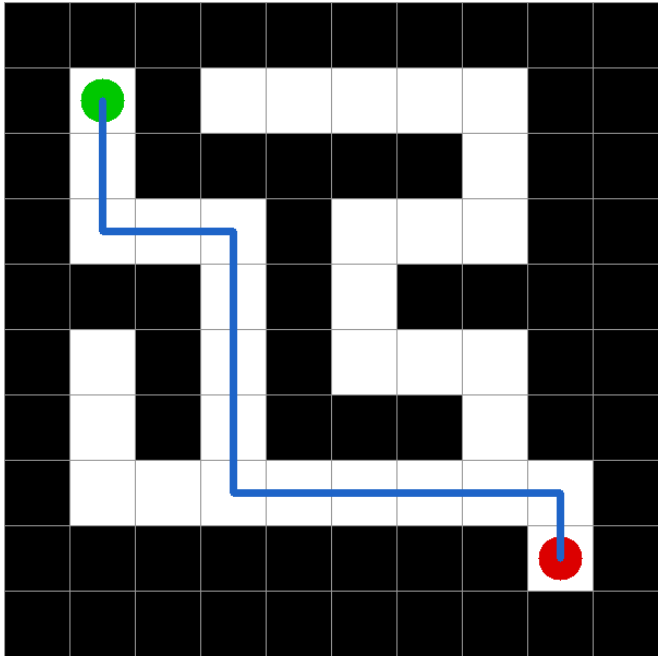
DQN needs reward shaping, replay buffer, and careful tuning.

Behavior Cloning reduces to supervised learning.

The trade-off: policy only generalizes to states seen during training.

Design goal: showcase the DL capability, not the training infrastructure.

Demo 1 — Clean maze



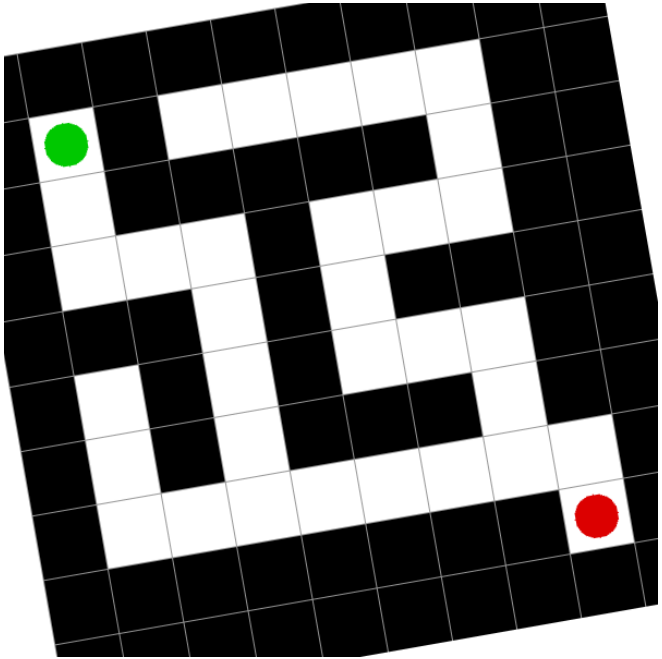
BOTH AGENTS SUCCEED

Non-DL
~4 milliseconds
Optimal path, deterministic

DL
~4 seconds
Optimal path, but ~1000× slower

On ideal inputs, classical wins on speed.

Demo 2 — Rotated maze



NON-DL COLLAPSES

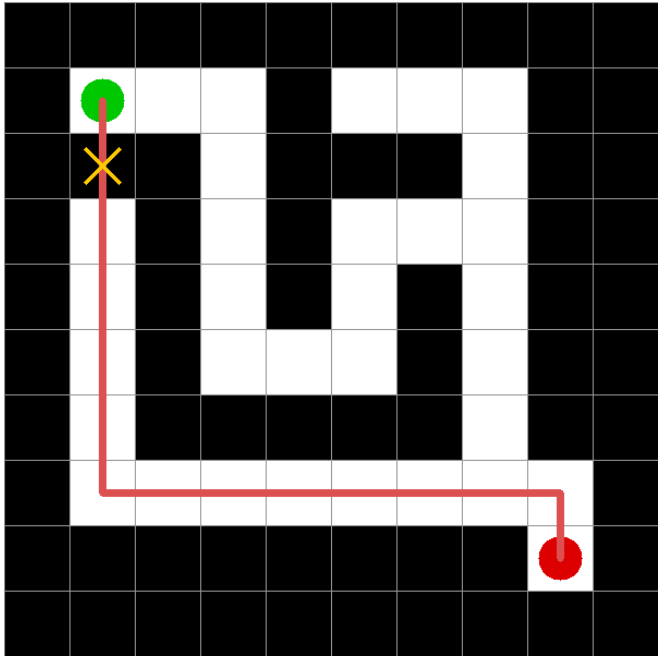
Perception: 80% cell accuracy
Success rate: **0%**

Why it fails

Center-pixel sampling assumes axis-aligned grid. A 10° rotation offsets every sample point by a few pixels — enough to misread whole columns.

Classical fails binary: works perfectly, or not at all.

Demo 3 — The surprise



CLEAN INPUT — DL STILL FAILS

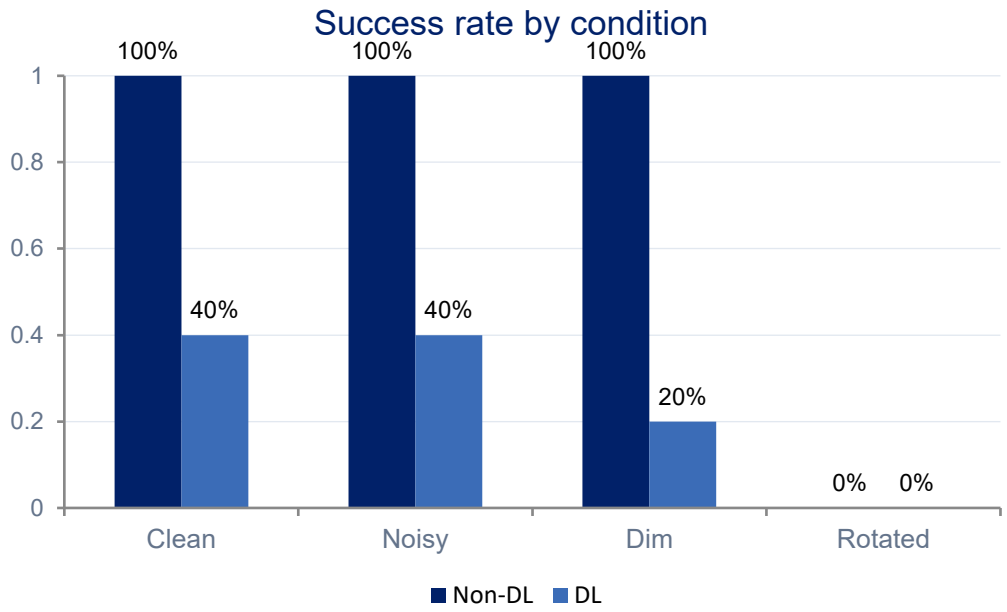
Perception: 92% cell accuracy
Success rate: **40%**

What happened

One misclassified wall cell (╳) creates a phantom corridor. The policy plans through it. In the real environment, the trajectory passes through solid wall → illegal.

This is cascade failure — and it's the thesis of this project.

Evaluation — 20 mazes, 4 conditions



KEY OBSERVATIONS

Non-DL perfect on 3/4 categories, 0% on rotation.

DL never exceeds 40% — even on clean inputs where perception is ~92%.

Both fail on rotation.

Why is DL's success rate so much lower than its perception accuracy?

The cascade failure

$$P(\text{success}) = 0.92^{20} \approx 19\%$$

In a modular pipeline, end-to-end success is

multiplicative across pipeline stages.

If perception is 92% correct per cell, and a plan traverses 20 cells, the probability of every cell being correct is ≈ 19%. Observed 40% is higher because BFS tends to route through interior cells where perception is more reliable.

92% sounds high, but it's not nearly high enough to drive a planner that trusts its input.

What this project taught

1

Two fragilities

Classical fails binary. DL fails in cascade. Different failure modes demand different mitigations.

2

Accuracy ≠ success

In modular pipelines, end-to-end success is multiplicative. Raw accuracy understates how much a pipeline degrades.

3

Hybrid wins

Production autonomous systems use DL perception + classical planning + validation. This project reproduced why that pattern exists.

IF REDESIGNED

Structured tool-use for VLM output · connectivity check between perception & planning · incremental plan-execute loop with re-perception

End of Document